

15/July/20

Date	/	/
Page		

Week-4 Module

* SECTION A :- Complexity Analysis of previous modules.

* Module 1 :- Pattern programs

① Square pattern

* Time Complexity :- $O(n^2)$

* Space Complexity :- $O(1)$

* Explanation :- The pattern uses two nested loops. The outer loop runs n times and the inner loop also runs n times, resulting in $n \times n$ operations. Therefore, the time complexity is $O(n^2)$. Only a few loop variables are used, so the space complexity is $O(1)$.

* Optimized Approach :- Since, every character must be printed $O(n^2)$ is the minimum required complexity and cannot be further optimized.

② Hollow Square pattern

+ Time Complexity :- $O(n^2)$

+ Space Complexity :- $O(1)$

+ Explanation :- The algorithm checks each position in an $n \times n$ grid and prints either a star or a space. Since every cell is visited once, the total operations are n^2 . No extra memory is required apart from loop processed.

+ Optimized Approach :- The complexity cannot be reduced because every position in the pattern must be processed.

③ Triangle pattern

+ Time complexity :- $O(n^2)$

+ Space complexity :- $O(1)$

+ Explanation :- The outer loop runs n times while the inner loop prints stars according to the row number. The total number of printed stars is approximately $n(n+1)/2$, which gives $O(n^2)$ complexity.

+ Optimized Approach 2 - No further optimization is possible because all characters must be displayed.

④ Inverted Triangle Pattern

+ Time complexity :- $O(n^2)$

+ Space complexity :- $O(1)$

+ Explanation :- The first row prints n stars, the second prints $n-1$, and so on. The total operations are $n(n+1)/2$, resulting in $O(n^2)$ time complexity.

+ Optimized Approach 2 - printing all characters is necessary, so the complexity cannot be improved.

⑤ Pyramid Pattern

+ Time complexity :- $O(n^2)$

+ Space complexity :- $O(1)$

+ Explanation :- The program uses nested loops to print spaces and stars for each row. The total number of operations grows proportionally to n^2 .

Optimized Approach 2 - Since each character must be printed, $O(n^2)$ is optimal.

⑥ Diamond pattern

Time Complexity :- $O(n^2)$
Space complexity :- $O(1)$
Explanation :- The diamond consists of an upper pyramid and a lower inverted pyramid. Both sections together require approximately n^2 operations.

Optimized Approach 1 - No better complexity is possible because the entire pattern must be generated.

⑦ Half Diamond Pattern

Time Complexity :- $O(n^2)$
Space complexity :- $O(1)$
Explanation :- The pattern first increases and then decreases the number of stars. The total number of printed characters is proportional to n^2 .

Optimized Approach 2 - Cannot be optimized further due to mandatory output generation.

⑧ Half Diamond Inverted Pattern.

- + Time Complexity :- $O(n^2)$
- + Space complexity :- $O(1)$
- + Explanation :- The algorithm prints rows in decreasing and increasing order. The total operations are proportional to n^2 .
- + Optimized Approach :- No further optimization is possible.

⑨ Number patterns (Square, increasing, Triangle, Diamond, Sandwich)

- + Time Complexity :- $O(n^2)$
- + Space Complexity :- $O(1)$
- + Explanation :- These patterns use nested loops to print numbers row by row. Since each element is printed exactly once, the total work is proportional to n^2 .
- + Optimized Approach :- The complexity cannot be reduced every number must be displayed.

* MODULE 2 :- Searching and Sorting

① Linear Search

- + Time Complexity :- $O(n)$
- + Space Complexity :- $O(1)$
- + Explanation :- Linear Search checks elements one by one until the target is found or the array ends. In the worst case, all n elements are examined.
- + Optimized Approach :- If data is sorted, Binary Search can be used to achieve $O(\log n)$ time complexity.

② Binary Search

- + Time Complexity :- $O(\log n)$
- + Space Complexity :- $O(1)$
- + Explanation :- Binary Search repeatedly divides the search space into two halves. Each iteration reduces the number of elements to search, resulting in logarithmic complexity.

+ Optimized Approach - Binary Search is already high efficient for sorted data.

③ Bubble Sort

+ Time Complexity - $O(n^2)$

+ Space Complexity - $O(1)$

+ Explanation - Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. Multiple passes are required, resulting in $O(n^2)$ complexity.

+ Optimized Approach - Merge Sort or Heap Sort can reduce the complexity to $O(n \log n)$.

④ Insertion Sort

+ Time Complexity - $O(n^2)$

+ Space Complexity - $O(1)$

+ Explanation - Insertion Sort inserts each element into its correct position in the sorted portion of the array. In the worst case, each insertion requires shifting many elements.

+ Optimized Approach 2- For large datasets, Merge Sort performs better with $O(n \log n)$ complexity.

⑤ Selection Sort

+ Time Complexity :- $O(n^2)$

+ Space Complexity :- $O(1)$

+ Explanation :- Selection Sort repeatedly finds the minimum element and places it at the correct position. The algorithm performs n^2 comparisons regardless of input order.

+ Optimized Approach :- Heap Sort and Merge Sort provide significantly better performance for large datasets.

★ MODULE 3 :- Advanced Sorting Algorithms.

⑥ Merge Sort

+ Time Complexity :- $O(n \log n)$

+ Space Complexity :- $O(n)$

+ Explanation 2- Merge Sort divides the array into smaller halves recursively and then merges them in sorted order. The division takes $\log n$ levels and merging requires $O(n)$ work at each level.

+ Optimized Approach 2- Merge Sort is already efficient and provides guaranteed $O(n \log n)$ performance.

② Quick Sort

+ Time Complexity 2- $O(n \log n)$ Average Case

+ Worst Case 2- $O(n^2)$

+ Space Complexity 2- $O(\log n)$

+ Explanation 2- Quick Sort selects a pivot element and partitions the array into smaller sections. On average, balanced partitions lead to $O(n \log n)$ performance.

+ Optimized Approach 2- Using a random pivot or median pivot reduces the chance of worst-case performance.

③ Heap Sort

- + Time Complexity $\sim O(n \log n)$
- + Space Complexity $\sim O(1)$
- + Explanation: Heap sort first builds a heap and then repeatedly extracts the largest element. Heap construction takes $O(n)$ time and each extraction takes $O(\log n)$.
- + Optimized Approach: Heap sort is already efficient and provides guaranteed $O(n \log n)$ performance without requiring extra memory.

★ SECTION B :- Analyze the Algorithm Complexity.

Week-4 Module

Problem Statement :-

1. Solⁿ

Code In C++ -

```
#include <iostream>
using namespace std;
```

```
int main() {
```

```
    int N;
```

```
    cin >> N;
```

```
    for (int i = 1; i <= N; i++) {
```

```
        cout << i;
```

```
        if (i != N)
```

```
            cout << " ";
```

```
    }
```

```
    return 0;
```

Java Code -

```
import java.util.Scanner;
```

```
public class Main {
```

```
    public static void main (String[] args) {
```

```
        Scanner sc = new
```

```
        Scanner (System.in);
```

```
        int N = sc.nextInt();
```

```
        for (int i = 1; i <= N; i++) {
```

```
            System.out.print(i);
```

```
            if (i != N) {
```

```
                System.out.print(" ");
```

```
        }
```

```
        sc.close();
```


Algorithm

1. Read the Input Value N .
2. Start a loop from 1 to N .
3. Print the current number.
4. Print a space after every number except the last one.
5. The loop ends after printing all numbers from 1 to N .

Time Complexity :- $O(N)$

The loop runs exactly N times, and each iteration prints one number. Since every iteration takes constant time, the overall time complexity is $O(N)$.

Space Complexity - $O(1)$

The program uses only a few variables & does not allocate any extra memory based on the input size.

More Efficient Approach

No more efficient approach exists because every number from 1 to N must be printed at least once. Hence, $O(N)$ is the optimal time complexity.

Example 1

Input

5

output

1 2 3 4 5

Explanation -

The no.'s from 1 to 5 are printed in increasing order.

Example 2

Input

8

output

1 2 3 4 5 6 7 8

Explanation -

The program prints every integer starting from 1 until 8 , maintaining increasing order.

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int N;
7      cin >> N;
8
9      for (int i = 1; i <= N; i++)
10     {
11         cout << i;
12
13         if (i != N)
14         {
15             cout << " ";
16         }
17     }
18
19     return 0;
20 }
```



input

5

1 2 3 4 5

...Program finished with exit code 0
Press ENTER to exit console.

Main.java

```
1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         Scanner sc = new Scanner(System.in);
7
8         int N = sc.nextInt();
9
10        for (int i = 1; i <= N; i++) {
11            System.out.print(i);
12
13            if (i != N) {
14                System.out.print(" ");
15            }
16        }
17
18        sc.close();
19    }
20 }
```

5
1 2 3 4 5

...Program finished with exit code 0
Press ENTER to exit console.

input

2. Problem Statement :

C++ code -

```
#include <iostream>
using namespace std;
```

```
int main() {
```

```
    int N;
```

```
    cin >> N;
```

```
    long long sum = 0;
```

```
    for (int i = 1; i <= N; i++) {
```

```
        [ Sum += i;
```

```
    ]
```

```
    cout << sum;
```

```
    return 0;
```

java code -

```
import java.util.Scanner;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new
```

```
Scanner (System.in);
```

```
        int N = sc.nextInt();
```

```
        long sum = 0;
```

```
        for (int i = 1; i <= N; i++) {
```

```
            [ Sum += i;
```

```
        ]
```

```
        System.out.print(sum);
```

```
        sc.close();
```


Time Complexity - $O(N)$

Space Complexity - $O(1)$

- the loop iterates from 1 to N , performing one addition in each iteration. therefore, the running time grows linearly with N .
- only a fixed number of variables (N , sum , and i) are used, regardless of the input size.

More Efficient Approach

Yes, instead of using a loop, the sum can be calculated directly using the mathematical formula:

$$Sum = \frac{N \times (N+1)}{2}$$

this approach reduces the time complexity to $O(1)$ while keeping the space complexity $O(1)$. However, since the problem specifically asks to use a loop, the above solution is appropriate.

Example 1

Input
5

Output
15

Explanation

the sum of the first five natural numbers is:

$$1 + 2 + 3 + 4 + 5 = 15$$

Example 2

Input
10

Output
55

Explanation

the sum of first ten natural no. is:

$$1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 = 55$$

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int N;
7      cin >> N;
8
9      long long sum = 0;
10
11     for (int i = 1; i <= N; i++)
12     {
13         sum += i;
14     }
15
16     cout << sum;
17
18     return 0;
19 }
```

5
15

...Program finished with exit code 0
Press ENTER to exit console.

input

Main.java

```
1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         Scanner sc = new Scanner(System.in);
7
8         int N = sc.nextInt();
9         long sum = 0;
10
11         for (int i = 1; i <= N; i++) {
12             sum += i;
13         }
14
15         System.out.print(sum);
16
17         sc.close();
18     }
19 }
```



```
5
15
...Program finished with exit code 0
Press ENTER to exit console.
```


Problem Statement :

3 soln

C++ code -

```
#include <iostream>
using namespace std;
```

```
int main () {
```

```
    int A, B;
```

```
    cin >> A >> B;
```

```
    long long powerValue = 1;
```

```
    for (int i = 0; i < B; i++) {
```

```
        powerValue *= A;
```

```
    }
```

```
    cout << powerValue;
```

```
    return 0;
```

Java code -

```
import java.util.Scanner;
```

```
public class Main {
```

```
    public static void main (String[] args) {
```

```
        Scanner sc = new
```

```
        Scanner (System.in);
```

```
        int A = sc.nextInt();
```

```
        int B = sc.nextInt();
```

```
        long powerValue = 1;
```

```
        for (int i = 0; i < B; i++) {
```

```
            powerValue *= A;
```

```
        }
```

```
        System.out.print (powerValue);
```

```
        sc.close();
```


Algorithm

1. Start.
2. Read two Integers A and B.
3. Initialize a variable `powerValue = 1`.
4. Repeat the following steps B times:
 - Multiply `powerValue` by A.
5. Print `powerValue`.
6. Stop.

Time Complexity: $O(B)$

The loop executes exactly B times, and each iteration performs one multiplication. Therefore, the running time increases linearly with the value of B.

Space Complexity: $O(1)$

The program uses only a fixed number of variables (A, B, `powerValue`, and `i`). No extra memory is allocated based on the input size.

More Efficient Approach -

Yes, A faster method is Binary Exponentiation which computes in $O(\log B)$ time instead of $O(B)$ by repeatedly squaring the base & reducing the exponent.

Example 1

Input

2 5

Output

32

Explanation

— the value of $2^{<\text{sup}>5^{<|\text{sup}>}}$ is:

$$2 \times 2 \times 2 \times 2 \times 2 = 32$$

Example 2

Input

3 4

output

81

Explanation

— the value of $3^{<\text{sup}>4^{<|\text{sup}>}}$ is:

$$3 \times 3 \times 3 \times 3 = 81$$


```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int A, B;
7      cin >> A >> B;
8
9      long long powerValue = 1;
10
11     for (int i = 0; i < B; i++)
12     {
13         powerValue *= A;
14     }
15
16     cout << powerValue;
17
18     return 0;
19 }
```



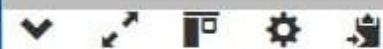
input

```
2 5
32
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```



```
1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         Scanner sc = new Scanner(System.in);
7
8         int A = sc.nextInt();
9         int B = sc.nextInt();
10
11         long powerValue = 1;
12
13         for (int i = 0; i < B; i++) {
14             powerValue *= A;
15         }
16
17         System.out.print(powerValue);
18
19         sc.close();
20     }
21 }
```



input

2 5
32

...Program finished with exit code 0
Press ENTER to exit console.